

Code-Sage AI Software Requirements Specification For the AI-Powered Technical-Debt Analytics Dashboard

Version 1.0

Group 16 | Project ID 7 | CS3203 - Software Engineering Project

Mentor: Mr. Anju Chamantha

Date: 9th August 2026

CodeSage AI	Version: 1.0
Software Requirement Specification	Date: 09/08/2026
CS3203-G16-SRS-v1.0	

Revision History

Date	Version	Description	Author
09/Aug/2026	1.0	The separate drafts prepared by the three authors were merged into a single document and reviewed for consistency across the functional requirements, the scoring model and the interface tables.	Group 16

Individual Contributions

Index No.	Name	Contributions
230432G	Nethmini R.M.N.	Prepared and documented Introduction, Functionality, Reliability Design Constraints, User Documentation, Purchased Components, Licensing, Legal, Copyright, and Other Notices, Applicable Standards.
230435T	Nethsara U.D.K.C.	Prepared and documented Overall Description, Functionality, Performance, Security, Database Requirements, Requirements traceability (Appendix A).
230451M	Pabasara H.H.J.	Prepared and documented Functionality, Usability, Supportability, Interfaces, Dashboard output definitions (Appendix B), Rule register (Appendix C).

CodeSage AI	Version: 1.0
Software Requirement Specification	Date: 09/08/2026
CS3203-G16-SRS-v1.0	

Table of Contents

1	Introduction	1
1.1	Purpose	1
1.2	Scope	1
1.3	Definitions, Acronyms, and Abbreviations	1
1.4	References	2
1.5	Overview	3
2	Overall Description	3
2.1	Product functions	3
2.2	User characteristics	3
2.3	Constraints.	4
2.4	Assumptions and dependencies	4
2.5	Requirements subsets	4
3	Specific requirements	4
3.1	Functionality	4
3.2	Usability	10
3.2.1	Learnability and training time	10
3.2.2	Task efficiency and low-noise presentation	11
3.2.3	Accessibility and standards conformance	11
3.2.4	Feedback, error recovery and consistency	12
3.3	Reliability	13
3.4	Performance	14
3.5	Security	15
3.6	Supportability	17
3.6.1	Coding standards and conventions	17
3.6.2	Modularity, configurability and extensibility	17
3.6.3	Testability and diagnosability	18
3.6.4	Machine-learning, documentation and release maintainability	18
3.7	Design Constraints	19
3.8	On-line User Documentation and Help System Requirements	19
3.9	Purchased Components	19
3.10	Interfaces	19
3.10.1	User Interfaces	19
3.10.2	Hardware Interfaces	21
3.10.3	Software Interfaces	21
3.10.4	Communications Interfaces	23
3.11	Database Requirements	23

CodeSage AI	Version: 1.0
Software Requirement Specification	Date: 09/08/2026
CS3203-G16-SRS-v1.0	

3.12	Licensing, Legal, Copyright, and Other Notices	26
3.13	Applicable Standards	27
4	Supporting Information	27
4.1	Appendix A - Requirements traceability	28
4.2	Appendix B - Dashboard output definitions	29
4.3	Appendix C - Rule register (rule id, severity, category, message template)	29
4.3.1	C.1 Rule-engine rules (source = rule)	29
4.3.2	C.2 SATD marker patterns (source = satd)	30

List of Tables

Table 1.3-1: Definitions, acronyms and abbreviations.	1
Table 2.2-1:User characteristics	3
Table 3.1-1:Functional requirement register (v1.0 and later releases)	4
Table 3.1-2:Severity and category assigned by each v1.0 rule	9
Table 3.1-3:The five factors of finding_priority	9
Table 3.2-1: Learnability and training-time requirements.	10
Table 3.2-2:Task-efficiency and low-noise requirements.	11
Table 3.2-3:Accessibility and standards-conformance requirements.	11
Table 3.2-4:Feedback, error-recovery and consistency requirements	12
Table 3.3-1:Reliability requirements.	13
Table 3.4-1: Performance requirements.	14
Table 3.5-1: Security requirements.	15
Table 3.6-1:Coding standards and conventions.	17
Table 3.6-2: Modularity, configurability and extensibility	17
Table 3.6-3: Testability and diagnosability.	18
Table 3.6-4: Machine-learning, documentation and release maintainability.	18
Table 3.7-1: Design constraints.	19
Table 3.8-1: Documentation and help requirements.	19
Table 3.10-1: Global user-interface elements present on every authenticated screen.	20
Table 3.10-2: Screen inventory: required panels, fields and states.	20
Table 3.10-3: Hardware interfaces and client-side prerequisites.	21
Table 3.10-4: External software interfaces.	21
Table 3.10-5: Internal software interfaces.	22
Table 3.10-6: Application REST endpoints required for v1.0.	22
Table 3.10-7: Communications interfaces.	23
Table 3.11-1: Database requirements.	23
Table 3.12-1: Licensing and legal requirements.	27
Table 3.13-1: Applicable standards.	27
Table 4.1-1: Requirements traceability matrix (extract).	28
Table 4.2-1: Dashboard Definitions	29
Table 4.3-1: Rule register: rule-engine rules.	29
Table 4.3-2: Rule register: SATD marker patterns.	30
Table 4.3-3: Debt-category taxonomy and dataset label mapping	30
Table 4.3-4: Risk badge template.	30

CodeSage AI	Version: 1.0
Software Requirement Specification	Date: 09/08/2026
CS3203-G16-SRS-v1.0	

1 Introduction

1.1 Purpose

The purpose of this document is to define the software requirements specification for the AI-Powered Technical Debt Analytics Dashboard. This document describes the overall functionality, scope, user characteristics, system constraints, and requirements of the platform. It provides a clear understanding of the system that will be developed to help small-scale agile software teams identify, analyze, and prioritize technical debt using static analysis and machine learning techniques

1.2 Scope

- Connect supported public GitHub repositories by URL.
- Run analysis with asynchronous worker queues to keep the UI responsive.
- Detect Self-Admitted Technical Debt (SATD) from source code comments.
- Perform static code analysis to identify code quality metrics and technical debt indicators.
- Predict bug-prone files using software metrics and machine learning.
- Provide a dashboard with overall code health and interactive file tree view to show hotspots in code.
- Produce prioritized list of "fix first" issues based on user-configurable category weighting profiles, SATD classification, and rule-based analysis.

1.3 Definitions, Acronyms, and Abbreviations

Table 1.3-1: Definitions, acronyms and abbreviations.

Term	Meaning
Technical debt (TD)	The implied future cost of shortcuts or poor design decisions taken in code.
SATD	Self-Admitted Technical Debt - debt a developer admits in natural language (for example "// TODO: temporary hack"). The literature recognises four sources (comments, commit messages, issues, pull requests); v1.0 detects SATD in source-code comments only (FR-9.1).
Finding	The atomic unit of output: one detected issue at a file:line:symbol, with a source, a category, a severity and a one-line reason.
source	Which detector produced a finding: rule satd .
category	What type of debt a finding is: code-design requirement documentation test security.
Rule engine	Deterministic thresholds over static metrics plus pattern-based security rules.
Risk model (ML-2)	Supervised classifier estimating a file's bug-proneness (0-1).
SATD classifier (ML-1)	Supervised NLP model classifying a source-code comment as debt and, if so, its category.
WMC	Method level Cyclomatic Complexity.
LOC	Lines Of Code.

CodeSage AI	Version: 1.0
Software Requirement Specification	Date: 09/08/2026
CS3203-G16-SRS-v1.0	

Term	Meaning
Churn	Recent change volume of a file, over a 90-day window anchored to the scanned commit's date (FR-11).
Process metrics	The four history-derived numeric features per file: churn, author count, file age, recency. Mined by PyDriller; consumed by ML-2 and by scoring (churn only).
Snapshot	The immutable stored result of one scan, keyed by repository, branch, commit SHA and timestamp.
Snapshot-scoped	Derived solely from the working tree at the scanned commit SHA.
Health score / grade	0-100 score and A-E grade summarizing a repository's or subtree's debt.
Scoring profile	The five per-category weights plus the trust slider s that balances rule and model confidence (FR-11, FR-20)
Visibility floor	The rule that critical security findings are never suppressed or down-weighted below visibility.
Workspace / tenant	The top-level data owner in the multi-tenant model; a project belongs to a workspace, not a person.
RBAC	Role-Based Access Control (org-admin / manager / developer / viewer).
RLS	PostgreSQL Row-Level Security, used to isolate tenants.
SZZ	Algorithm that links bug-fixing commits back to the changes that induced them (used to label defective files).
D'Ambros	D'Ambros bug prediction dataset, used for risk-model training and evaluation. Offline training data only.
MSW	Mock Service Worker - intercepts network calls to serve mock API data during frontend development and testing.
FR / NFR	Functional / Non-Functional Requirement.
SAD / SDD	Software Architecture Document / Software Design Document.

1.4 References

- [1] M. Aniche, "CK: Code metrics for Java," GitHub. <https://github.com/mauricioaniche/ck> [Accessed: Aug. 7, 2026].
- [2] D. Spadini, *PyDriller: A Python Framework for Mining Software Repositories*. GitHub repository. [Online]. Available: <https://github.com/ishepard/pydriller>. [Accessed: Jul. 30, 2026].
- [3] M. Brunsfeld et al., *Tree-sitter: An Incremental Parsing System for Programming Tools*. GitHub repository. [Online]. Available: <https://github.com/tree-sitter/tree-sitter>. [Accessed: Jul. 30, 2026].
- [4] The Scikit-learn Developers, *Scikit-learn*. GitHub repository. [Online]. Available: <https://github.com/scikit-learn/scikit-learn>. [Accessed: Jul. 30, 2026].
- [5] S. Ramírez, *FastAPI*. GitHub repository. [Online]. Available: <https://github.com/fastapi/fastapi>. [Accessed: Jul. 30, 2026].
- [6] Redis Ltd., *Redis: In-memory data structure store*. GitHub repository. [Online]. Available: <https://github.com/redis/redis>. [Accessed: Jul. 30, 2026].
- [7] Celery Project, "Celery: Distributed task queue," GitHub repository. [Online]. Available: <https://github.com/celery/celery>. [Accessed: Jul. 30, 2026].
- [8] PostgreSQL Global Development Group, "PostgreSQL: The World's Most Advanced Open Source Relational Database," GitHub repository. [Online]. Available: <https://github.com/postgres/postgres>. [Accessed: Jul. 30, 2026].

CodeSage AI	Version: 1.0
Software Requirement Specification	Date: 09/08/2026
CS3203-G16-SRS-v1.0	

- 30, 2026].
- [9] Vercel Inc., "Next.js: The React framework for the web," GitHub repository. [Online]. Available: <https://github.com/vercel/next.js>. [Accessed: Jul. 30, 2026].
- [10] Tailwind Labs, Inc., "Tailwind CSS: A utility-first CSS framework for rapid UI development," GitHub repository. [Online]. Available: <https://github.com/tailwindlabs/tailwindcss>. [Accessed: Jul. 30, 2026].
- [11] S. Kumar Pal, "Coding Standards and Guidelines," GeeksforGeeks, May 23, 2024. [Online]. Available: <https://www.geeksforgeeks.org/software-engineering/coding-standards-and-guidelines/>. [Accessed: Jul. 30, 2026]
- [12] M. D'Ambrosio, M. Lanza, and R. Robbes, "An extensive comparison of bug prediction approaches," in Proceedings of the 7th IEEE Working Conference on Mining Software Repositories (MSR), 2010, pp. 31–41.
- [13] E. Sutoyo, "Replication Package of Deep Learning and Data Augmentation for Detecting Self-Admitted Technical Debt," edisutoyo/satd-augmentation GitHub repository, 2026. [Online].

1.5 Overview

The project proposes an AI-Powered Technical Debt Analytics Dashboard for small-scale agile software teams; delivered as a multi-tenant web-based SaaS platform. The platform allows the teams to identify, analyze and prioritize technical debt. Teams connect their GitHub repositories, and the system will analyze the source code and code comments of the selected repository to identify Self-Admitted Technical Debt (SATD) using static analysis and machine learning. It will output a code health score, categorize the identified technical debt, predict the bug proneness and prioritize the issues based on user-configurable category weighting profiles, SATD classification and rule-based analysis. This allows developers to understand the current condition of their codebase and identify the areas that require immediate attention.

2 Overall Description

2.1 Product functions

The system provides the following main functions.

- Authenticate users through GitHub and allow them to connect public GitHub repositories for analysis.
- Extract static software metrics and repository process metrics from the selected revision of the repository.
- Detect rule-based code-design and security-related technical-debt findings.
- Predict file-level bug proneness using both process related metrics and software metrics with machine learning models.
- Prioritize detected findings using their severity, debt category importance, file-change activity and predicted bug risk, determined according to the active scoring profile.
- Display health score of the repository, prioritized findings, category distributions, file hotspots and historical trends of repository health through an interactive dashboard.
- Store the results of successfully completed analyses so users can review repository changes over time.
- Allow users to configure category weights and rule vs. model trust without running another analysis.

2.2 User characteristics

Table 2.2-1: User characteristics

User class	Description	Primary needs
Developer/ Viewer/ Stakeholder	A software developer who uses the system to identify and prioritize technical debt	Connect and analyze a repository with minimal configurations, receive clear and prioritized findings, and determine which technical-debt issues should be addressed

CodeSage AI	Version: 1.0
Software Requirement Specification	Date: 09/08/2026
CS3203-G16-SRS-v1.0	

	of a repository.	first while balancing debt remediation with feature development and release commitments.
--	------------------	--

2.3 Constraints.

- v1.0 analyses Java source files (.java) only, because CK is a Java only metric extractor. The software metrics and rule-based findings available in the initial release shall be limited to those that can be extracted or detected reliably using the selected analysis tools.
- In the initial release, the Self-Admitted Technical Debt (SATD) detection shall be limited to source-code comments. Commit messages, issues and pull requests shall not be used as SATD detection inputs.
- The machine learning models shall be trained and evaluated using publicly available datasets. Therefore, input features for the machine learning models and predictive performance shall be limited by the quality, representativeness of the available data.

2.4 Assumptions and dependencies

- Users have valid GitHub accounts and can authenticate through the supported GitHub authentication mechanism.
- GitHub services required for authentication, repository metadata retrieval and repository cloning remain available.
- Connected repositories contain source code written in supported programming languages and sufficient accessible Git history for the required static and process metric extraction.

2.5 Requirements subsets

- Core Functional Requirements – v1.0
 - Include: GitHub repository integration, static analysis of source codes, technical debt detection, self-admitted technical debt detection from source code's comments, file level bug-proneness prediction, finding prioritization, dashboard visualization, configurable scoring profiles and analysis history.
- Planned Enhancements – v1.1
 - Include: marking findings as accepted debt, resolved or false positives and improved accessibility for complex dashboard components.
- Future Platform Features – v2
 - Include multi user workspaces, organization administration, team-member management, role-based access control and workspace level repository authorization.

3 Specific requirements

3.1 Functionality

Notation: each requirement is written as FR-n [release] Name. "The system" means Code Sage AI.

Table 3.1-1: Functional requirement register (v1.0 and later releases)

ID	Release	Requirement	Summary
FR-1	v1.0	Authentication and session	Sign in with GitHub; authenticated session; land on Projects; sign out from the Account menu. Session state is managed per request (token), never held in server memory.

CodeSage AI	Version: 1.0
Software Requirement Specification	Date: 09/08/2026
CS3203-G16-SRS-v1.0	

ID	Release	Requirement	Summary
FR-2	v1.0 / v2 full	Workspace and tenant isolation	Every project scan and finding belongs to exactly one workspace. In v1.0, each workspace shall have one active user and shall not provide roles or multi-user collaboration features. The tenant ownership structure shall be included from the initial database design so that multi-user workspaces and role-based access control can be introduced in a later release.
FR-3	v1.0	Connect a repository (public repositories by URL paste)	Add a project (a repository) by pasting a public repository URL. The system validates the URL and records name, owner, visibility and default branch.
FR-4	v1.0	Projects list and selection	Display connected projects vertically with name, owner, visibility and current health status (score, grade, delta); Select sets the active project, which populates the Dashboard and Scan-History records.
FR-5	v1.0	Branch selection	Branch dropdown in the dashboard top navigation panel, default branch automatically selected. Analysis is per-branch: each branch has its own snapshots and trend.
FR-6	v1.0	Repository scan (manual, asynchronous, cancellable)	Scan control runs the pipeline on an asynchronous worker: idle → queued → running NN% (with Stop) → done, error or cancelled. Cancel during scan remains the previous snapshot intact. The system may skip re-scanning when the branch head SHA equals the last scanned SHA.
FR-7	v1.0	Metrics extraction	From a local clone at the selected branch's commit SHA: static metrics with CK; process metrics with PyDriller (churn, author count, file age, recency); and source-code comments extracted with Tree-sitter from the working tree at that SHA for the SATD classifier.
FR-7.1	v1.0	Extraction boundary	Git history enters the processing pipeline as numbers, never as text. Commit messages, pull requests, issues and previously stored snapshots are not detection inputs. Consequence: a scan only depends on the repository(branch) at that SHA, a fixed model version.
FR-8	v1.0	Detection - rule engine	Deterministic thresholds plus security patterns (hardcoded secret by regex and entropy, SQL string concatenation). Each finding records file, line, symbol, category, severity, measured value, threshold and rule id.
FR-8.1	v1.0	Severity and category assigned at detection	Every rule definition carries a fixed category and a fixed severity alongside its message template; both are written onto the finding at detection and are never recomputed by scoring. Re-scoring embedded to a set or selected by the user. The SATD classifier predicts categories only. Its severity comes from the marker table in FR-9.2; the risk model assigns neither. No ML model or user assigns severity. Severity answers "how bad is this kind of problem" (the same answer for every team) while the profile's weight answers "how much does this team care" (different per team). The register is in Appendix C.

CodeSage AI	Version: 1.0
Software Requirement Specification	Date: 09/08/2026
CS3203-G16-SRS-v1.0	

ID	Release	Requirement	Summary
FR-8.2	v1.0	source has exactly two values	source for each finding shall be rule or satd. Security patterns execute inside the rule engine, so a security finding is source = rule with category = security. The risk model does not emit findings, so no finding can be originated by risk model.
FR-9	v1.0	Detection - SATD classifier (ML-1)	Classify each extracted source-code comment as debt or not and, if debt, assign a category from {code-design, requirement, documentation, test} (FR-9.3), using a supervised NLP model trained on the SATDAUG dataset. Each finding is mapped to the comment's file:line and the comment. The model predicts category only; severity is assigned per FR-9.2.
FR-9.1	v1.0	SATD scope - the comments file only	v1.0 trains, validates and evaluates on SATD dataset data-augmentation-code_comments.csv only. Commit messages, issues and pull requests are not used in v1.0 release.
FR-9.2	v1.0	SATD severity – comment regex patterns	After the classifier determines that a comment is debt and assigns its category, severity is assigned by matching the comment text. FIXME/BUG/XXX/BROKEN -> high; TODO/HACK/TEMP/WORKAROUND -> medium; NOTE/REVIEW/NIT -> low; no marker -> medium. Patterns are evaluated high to low, highest match wins, and match anywhere in the comment. A supervised model can predict only what its training data labels, and the dataset labels categories, not severities. So, severity cannot be learned and must be deterministic. The patterns and templates are in Appendix C.2.
FR-9.3	v1.0	Debt-categories and label mapping	The categories are fixed by the labels in the comments file (68,514 labelled comments): code-design <- code/design_debt (2,703), requirement <- requirement_debt (2,271), test <- test_debt (2,635), documentation <- documentation_debt (2,701); plus security, which is not in the dataset and is emitted only by the rule engine. non_debt (58,204) is the negative class of the debt/not-debt decision and is never a category.
FR-10	v1.0	Detection - risk model (ML-2)	Per-file bug-proneness score (0-1) from a supervised classifier over CK product metrics plus the four PyDriller process metrics, trained on the D'Ambros defect dataset. It does not assign category or severity. It produces no findings. It has exactly two effects: it boosts the priority of the findings in that file through the bounded risk_factor multiplier (FR-11), and it appears as a per-file risk level for the finding where it belongs. A risky file with no findings contributes no debt - every point of debt traces to a finding the user can open.

CodeSage AI	Version: 1.0
Software Requirement Specification	Date: 09/08/2026
CS3203-G16-SRS-v1.0	

ID	Release	Requirement	Summary
FR-11	v1.0	Scoring and prioritization	A mathematical function over stored findings using the active profile. $\text{finding_priority} = \text{base_points}(\text{severity}) \times \text{category_weight}[\text{category}] \times \text{source_trust}(\text{finding}) \times \text{churn_factor}(\text{file}) \times \text{risk_factor}(\text{file})$; $\text{file_debt} = \text{sum of open finding priorities}$; $\text{repo_health} = 100 \times (1 - \min(1, \text{sum file_debt} / (k \times \text{KLOC})))$; grade $A \geq 85$, $B \geq 70$, $C \geq 55$, $D \geq 40$, $E < 40$. $\text{churn_factor} = 1 + \min(\text{commits_90d}, 20) / 20$ (1.0-2.0); $\text{risk_factor} = 1 + \text{ml_trust} \times \text{risk_score}$ (1.0-2.5); $\text{rule_trust} = 0.5 + s$, $\text{ml_trust} = 1.5 - s$, and source_trust is 1.0 for the security category, rule_trust for rule findings and ml_trust for SATD findings. Base points are a lookup over the stored severity. The 90 day churn window is considered from the scanned commit's committer date. k shall be calibrated against a set of reference repositories before release; the calibrated value and the calibration method shall be recorded in the SAD.
FR-12	v1.0	Overall Health card	Health score (0-100), grade (A-E), delta (difference) compared to the previous snapshot and a count of critical/high issues, for the selected branch.
FR-13	v1.0	Category breakdown pie	Pie chart of technical debt by category, computed from stored findings.
FR-14	v1.0	Health trend chart	Health per scan over time for the selected branch, derived from the stored snapshots. Every point is computed under the currently active profile, so selecting a different profile redraws the entire history under that profile and every point stays comparable. The chart is labelled with the active profile name. ("custom" label for user set profiles)
FR-15	v1.0	Refactor-First list and debt-type filter	Prioritized list of rule engine and SATD findings sorted by priority; each finding shows source, category, severity, file:line and the one-line reason. The badge renders the stored severity (FR-8.1); the user cannot influence severity on his own. The user can filter by debt type.
FR-16	v1.0	One-line reason (deterministic templates)	Every finding carries a one-line plain-English reason generated from string templates with the finding's own values interpolated.
FR-17	v1.0	Finding detail - in-place detail mode	Selecting a finding shall switch the dashboard into detail mode, rendered in place rather than as an overlay: the region holding the health card and trend chart is replaced by the finding's evidence, one-line reason and file:line:symbol and code snippet; the file tree auto-expands and highlights that finding's file; and the Refactor-First list condenses so the user can move between findings. Closing restores the cards. The selected finding is reflected in the URL. FR-17b [v1.1]: accept-debt / resolve / false-positive actions - v1.0 is view-only.

CodeSage AI	Version: 1.0
Software Requirement Specification	Date: 09/08/2026
CS3203-G16-SRS-v1.0	

ID	Release	Requirement	Summary
FR-18	v1.0	Hotspot file-tree heat map	Interactive file tree colored red to green by health with a per-file risk score, folders aggregating their children, expand/collapse and drill-in those re-aggregates from stored file scores without re-scanning. When the dashboard enters detail mode the tree automatically expands the ancestors of the selected finding's file, scrolls it into view and highlights it, remaining fully interactive throughout.
FR-19	v1.0	Scan history	List past snapshots for the active project and branch (date, commit SHA, health score, grade, delta, finding count); selecting one loads that snapshot into the dashboard read-only.
FR-20	v1.0	Scoring profiles - presets and user adjustable category weights	A profile comprises five category weights for security, code design, requirement, documentation, and test, which are constrained between 0.1 and 3.0 to prevent unbounded values from driving every repository to a failing grade against the calibration constant k. It also includes a trust slider defaulting to 0.5 within a 0 to 1 range. Users can instantly initialize or reset these values using three presets, with Balanced serving as the workspace default. Applying a profile instantly recalculates stored findings without triggering a new scan, pipeline execution, or snapshot creation, as snapshots are strictly bound to commit SHAs rather than profile states. Additionally, profiles explicitly exclude severity metrics. A profile is applied in a single idempotent write carrying the complete profile with all five weights and the trust slider. Dragging a control changes client state only; no request is issued until the user confirms with Apply button. The server clamps every weight on write and returns the stored profile, so the client confirms what was saved rather than assuming its own values were accepted. The active profile is server-side state scoped to the workspace, so the read endpoints are unchanged and carry no profile parameter.
FR-21	v1.0	Snapshot persistence - facts stored, scores derived	Store the result of every scan as an immutable snapshot keyed by repository, branch, commit SHA and timestamp. A snapshot stores facts about the code at that commit - findings with their evidence, the per-file risk score and the raw commit counts from which the churn factor is computed, the tree, the commit SHA, the scan time, the finding count and the model version. Every score - priority, file debt, health score, grade, delta, category breakdown - is a function of the active profile and is therefore derived on read, never stored. Services remain stateless.
FR-22	v1.0	Account menu	Account control at the foot of the left navigation panel exposing sign out, with settings for theme switch (dark/light).
FR-23	v2	Team and RBAC	Multi-user workspaces with org-admin / manager / developer / viewer roles, invitations and auto-shared projects; repository access uses the workspace's GitHub App installation.

CodeSage AI	Version: 1.0
Software Requirement Specification	Date: 09/08/2026
CS3203-G16-SRS-v1.0	

ID	Release	Requirement	Summary
FR-24	v1.0	Critical-security visibility floor	Critical security findings are guaranteed to remain visible regardless of the active profile through three specific mechanisms. First, severity is not user-configured, ensuring classifications like hardcoded secrets remain permanently critical within the rule register. Second, security findings bypass the trust slider by maintaining a constant source trust of 1.0, preventing the trust variable from de-weighting them. Third, the system explicitly pins critical security findings to the visible list, overriding computed priorities even when the security weight is set to its 0.1 minimum.
FR-25	supporting	ML evaluation versus rule baseline	The SATD classifier and risk models are evaluated using precision, recall, and F1 scores, alongside AUC for the risk model. These metrics are benchmarked strictly against a deterministic rule baseline rather than previous research papers.

Table 3.1-2: Severity and category assigned by each v1.0 rule

Rule id	Trigger	Category	Severity
complex-function	WMC > 15	code-design	Medium
long-method	function > 80 LOC	code-design	Medium
deep-nesting	nesting > 4	code-design	Medium
large-file	file > 800 LOC	code-design	Low
hardcoded-secret	regex and entropy	security	Critical
sql-concat	SQL string concatenation	security	High

Severity assignment is strictly deterministic across all sources in the initial release, ensuring that machine learning models never assign severity levels. The rule engine and security patterns assign severity based on predefined rules. The SATD classifier predicts only categories (FR-9), while severity is assigned relying on regex patterns (FR-9.2). Additionally, the risk model (FR-10) does not generate individual findings or assign labels at all.

Table 3.1-3: The five factors of finding priority.

Factor	Question it answers	Owner	Range
base_points(severity)	How bad is this kind of problem?	System - the rule register (FR-8.1) or the marker table (FR-9.2)	1 / 3 / 5 / 8
category_weight[category]	How much does this team care about this type of debt?	User - 5 sliders (FR-20)	0.1 - 3.0
source_trust(finding)	How much does this team trust the rules versus the model?	User - trust sliders (FR-20)	0.5 - 1.5; always 1.0 for security

CodeSage AI	Version: 1.0
Software Requirement Specification	Date: 09/08/2026
CS3203-G16-SRS-v1.0	

Factor	Question it answers	Owner	Range
churn_factor(file)	How actively is this file being changed?	Evidence - PyDriller	1.0 - 2.0
risk_factor(file)	How bug-prone is this file?	Model - ML-2 (FR-10)	1.0 - 2.5

The scoring mechanism activates upon the completion of a scan or a change in the active profile. It utilizes a pure function to calculate scores by processing the active profile, which includes five category weights and a trust slider designated as 's'. Because weights are applied during the scoring phase, modifying a profile never requires a new scan.

The system computes finding priority, individual file debt, repository health scores, and grades as specified in FR-11. These calculations rely on fixed severity base points of 8 for Critical, 5 for High, 3 for Medium, and 1 for Low. To guarantee that rescanning the same commit SHA consistently produces identical results, the 90-day churn window is measured from the exact committer date of the analyzed commit. In accordance with FR-21, all outputs are dynamically generated upon reading the stored findings under the active profile. These outputs contain finding priority, per-file debt scores, repository health scores, grades, deltas, and category breakdowns.

3.2 Usability

Usability is this product's core differentiating attribute against competing tools. They are technically capable but overwhelm small teams with high noise and heavy configuration. The requirements listed below make the system testable for low noisiness. Each requirement states a measurable target and the method by which it will be verified.

3.2.1 Learnability and training time

Table 3.2-1: Learnability and training-time requirements.

ID	Requirement	Measurable target	Verification
U-1	A first-time developer shall complete the primary journey - sign in, connect a public repository, run a scan and open the top-priority finding - with no training and no documentation.	At least 4 of 5 test participants succeed without guidance; median completion time $\leq$ 5 minutes.	Usability test with 5 participants.
U-2	No formal training shall be required for any user class (v2.0 will have user classes)	viewer/stakeholder can identify dashboard outputs (trend-charts, scores) with self guidance .	Same usability-test session.

CodeSage AI	Version: 1.0
Software Requirement Specification	Date: 09/08/2026
CS3203-G16-SRS-v1.0	

3.2.2 Task efficiency and low-noise presentation

Table 3.2-2: Task-efficiency and low-noise requirements

ID	Requirement	Measurable target	Verification
U-3	On an already-scanned project, each core task shall complete within a stated budget measured from dashboard load.	Identify the highest-priority issue ≤ 30 s and ≤ 2 clicks; open finding detail 1 click; filter by debt type ≤ 2 interactions; switch branch ≤ 2 ; load a past snapshot ≤ 2 .	Automated end-to-end tests assert the click paths.
U-4	The dashboard shall present findings ranked by priority and shall never present an unranked dump of raw results.	The initial Refactor-First view shows at most 10 findings, sorted by priority descending; everything else is reached by explicit expanding/filtering.	Component test on filtering.
U-5	A non-technical stakeholder shall correctly interpret the health score, grade, delta direction and trend direction without assistance.	At least 4 of 5 non-technical participants interpret all four correctly.	Comprehension questions in the usability test.
U-6	Every finding shall be explained in exactly one English sentence naming the symbol and, for rule findings, the measured value and the threshold crossed; for SATD findings, quoting the comment and its predicted category.	100% of findings; ≤ 140 characters per reason.	Review of the reason-template table (Appendix C).

3.2.3 Accessibility and standards conformance

Table 3.2-3: Accessibility and standards-conformance requirements.

ID	Requirement	Measurable target	Verification
U-7	The user interface shall conform to WCAG 2.1 Level AA for contrast, including every severity badge and every heat-map tint.	Contrast ratio $\geq 4.5:1$ for body text and $\geq 3:1$ for user-interface components and graphical objects; 0 violations.	Automated accessibility audit (axe-core / Lighthouse).
U-8	Color shall never be the sole carrier of meaning.	Heat-map tint is always accompanied by a numeric health score and letter grade; severity is always a text-labelled badge, never a colored dot alone; 0 color-only signals.	Manual audit plus greyscale review.

CodeSage AI	Version: 1.0
Software Requirement Specification	Date: 09/08/2026
CS3203-G16-SRS-v1.0	

ID	Requirement	Measurable target	Verification
U-9	Every interactive component must be fully operable using only a keyboard. The system must enforce a sequential, logical tab order and display a permanent visual indicator so users always know which element is currently active.	This standard applies to 100% of the interface. Mandatory components include navigation links, branch selectors, scan/stop commands, data filters, finding rows, file-tree nodes, and panel closing mechanisms.	Keyboard-only walkthrough plus automated accessibility audit.

3.2.4 Feedback, error recovery and consistency

Table 3.2-4: Feedback, error-recovery and consistency requirements.

ID	Requirement	Measurable target	Verification
U-10	Any action that takes longer than one second shall show progress within that second.	Scans surface a live state machine (idle, queued, running NN%, done, cancelled or error) with a Stop control and a terminal notification; data loads show skeleton placeholders rather than blank regions; no unacknowledged action exceeds 1 s.	End-to-end test of the scan state machine.
U-11	Error recovery shall be non-destructive: a failed or cancelled scan leaves the previous snapshot displayed and usable, and every error message states what failed and the next action.	0 dead-end error states.	End-to-end cancel and failure paths
U-12	Presentation shall be consistent: one design system, one rendering per severity and grade on every screen, and user-interface copy that uses the exact terms defined in Section 1.3.	0 divergent design tokens; all colors resolved from CSS variables; no synonyms for "finding", "snapshot" or "debt category".	Code-review
U-13	The application shall be fully usable from a laptop viewport upwards, with no horizontal scrolling of the dashboard.	Minimum supported viewport 1280 x 720; the two-column dashboard collapses to a single column below the large breakpoint.	Automated viewport test.
U-14	Every list shall show a named empty state until it populates.	100% of list views (for example "No repositories yet - connect one to see its health.").	Screen-by-screen review.

CodeSage AI	Version: 1.0
Software Requirement Specification	Date: 09/08/2026
CS3203-G16-SRS-v1.0	

3.3 Reliability

Table 3.3-1: Reliability requirements.

ID	Requirement	Verification Criteria
REL-01	System Availability	The production deployment of the system shall be designed to support and maintain a monthly availability target of at least 99%. This does not include maintenance and unavailability caused by required external services.
REL-02	Isolation of Failures	A failure during the analysis of a single repository shall not interrupt or block other analysis jobs, nor prevent users from accessing results from previously completed analyses. The failure shall also not prevent the initiating user from starting a new analysis of the same repository.
REL-03	Analysis History	All the analysis attempts shall be recorded by the system as a history. It shall include the status, associated repository revision, start time and complete time of each analysis.
REL-04	Analysis Job Recovery	If a failure happens during an analysis job due to a transient processing or communication failure, the system shall retry at least three times before marking the analysis job as failed.
REL-05	Integrity of Analysis Results	Results of a new analysis shall be saved to the database only after all analysis stages have successfully completed. A failed, cancelled or partially completed analysis attempts shall not create finalized analysis snapshot. However, its status, timestamps, retry information and diagnostic details shall be retained in the analysis history.
REL-06	External Service Failure Handling	When a required external service like GitHub is unavailable or returns an error, the system shall report the relevant operation as unsuccessful and shall not present incomplete analysis results as successfully completed results.
REL-07	Recovery time	After a recoverable failure of the system or one of its components, normal system service shall be restored under thirty minutes without loss or corruption of previously finalized data. This does not include the recovery time for external services outside the system's control.
REL-08	Mean Time Between Failures	The production deployment of the system shall target a mean time between service-impacting failures of at least 500 operational hours. A service-impacting failure is a failure of a system component that makes a core system function unavailable.
REL-09	Persistent Data Integrity	A failure of a component of the system or external service shall not corrupt or erase previously stored user data, repository information, configuration, analysis history or finalized analysis results.
REL-10	Static Analysis Consistency	If the analysis version of the static analysis tool, repository revision, rule definitions and rule thresholds remain unchanged, repeated static analysis on the same repository version shall output identical static analysis metrics and deterministic rule-based findings.
REL-11	SATD Classification Evaluation	The SATD classification machine learning model shall be evaluated on a held-out test set using precision, recall and F1-

CodeSage AI	Version: 1.0
Software Requirement Specification	Date: 09/08/2026
CS3203-G16-SRS-v1.0	

ID	Requirement	Verification Criteria
		score. These metrics shall be documented for each model version before deployment including overall and per-class results, test-set size and class distribution. The target F1-score for binary SATD detection (debt / non-debt) detection shall be 0.80.
REL-12	Bug-Proneness Prediction Accuracy	The bug-proneness prediction model shall achieve an ROC-AUC score of at least 0.70 on the held-out evaluation dataset. Precision, recall, F1-score and ROC-AUC shall be recorded and documented for each model version before deployment.
REL-13	Defect Severity Classification	Software defects shall be categorized as critical, significant or minor according to their impact on system operation. If a defect causes one or more of the following, it is classified as a critical debt. 1. Unauthorized access to data that belongs to another tenant. 2. Loss or corruption of user or analysis data. 3. Authentication or authorization bypass affecting protected system resources. 4. Complete unavailability of a core system function. 5. Invalid or incomplete analysis results are shown as successfully completed results. If a defect prevents a major system function but allows the system and its other core functionality continue, is classified as a significant defect. A minor defect is a defect with limited functional impact that does not prevent completion of a core system workflow.
REL-14	Maximum Outstanding Critical Defects	A production release of the system shall contain no known unresolved critical defects.

3.4 Performance

Table 3.4-1: Performance requirements.

ID	Requirement	Verification Criteria
PERF-01	Immediate User Interface Feedback	The system shall give visible feedback for a direct user interaction within 0.1 seconds indicating that the interaction has been received.
PERF-02	Interactive Response Time	The user interactions that do not involve a repository retrieval, analysis shall complete within 1 second.
PERF-03	Analysis Request Submission	When a user initiates an analysis, the analysis job shall be placed in the queue within 1 second.
PERF-04	Long-Running Operation	For an operation that is expected to take more than 10 seconds,

CodeSage AI	Version: 1.0
Software Requirement Specification	Date: 09/08/2026
CS3203-G16-SRS-v1.0	

ID	Requirement	Verification Criteria
	Feedback	the system shall provide its progress and status information until the operation completes, fails or is cancelled.
PERF-05	Non-Blocking Analysis Processing	Repository retrieval, analysis, technical debt detection and prioritization shall be performed without preventing the users from interacting with the user interface of the application.
PERF-06	Baseline System Capacity	The system shall support at least 50 single-user workspaces, giving a baseline capacity of 50 registered users, while meeting the response-time requirements.
PERF-07	Concurrent Analysis Processing	The system shall support at least three repository analysis requests concurrently. When analysis requests exceed the currently available processing capacity, additional requests shall be queued without blocking interactive system functionality.
PERF-08	Degraded Operation Under High Load	When analysis demand exceeds the available processing capacity, the system shall continue to provide authentication, navigation, access to previously completed analysis results, and access to analysis history while pending analysis jobs remain queued.
PERF-09	External Service Degradation	When a required external repository service is unavailable, rate-limited, or experiencing degraded performance, operations dependent on that service may be delayed or queued, while system functionality that does not depend on the affected external service shall remain available.
PERF-10	Resource Monitoring	The system shall monitor the utilization of critical computing resources, including processor usage, memory usage, persistent storage, database connections, and analysis-worker queue utilization, to support detection of resource exhaustion and performance degradation.

3.5 Security

Table 3.5-1: Security requirements.

ID	Requirement	Verification Criteria
SEC-01	User Authentication	The system shall require users to authenticate before accessing protected workspace data, repository configurations, scoring profiles, analysis history or analysis results.
SEC-02 [v2]	Role-Based Access Control	The system shall enforce role-based access control so that authenticated users can perform only the operations and access only the resources permitted by their assigned roles within their organization.
SEC-03	Tenant Data Isolation	The system shall ensure that users belonging to one tenant cannot access, modify, or delete data belonging to another tenant, including repository information, source code obtained for analysis, configurations, analysis history, and analysis results.
SEC-04 [v2]	Repository Authorization	The system shall access a private repository only when the corresponding workspace has explicitly authorized access to that repository. The system

CodeSage AI	Version: 1.0
Software Requirement Specification	Date: 09/08/2026
CS3203-G16-SRS-v1.0	

ID	Requirement	Verification Criteria
		shall not access repositories outside the granted authorization scope.
SEC-05	Least-Privilege Repository Access	The system shall use read-only operations to retrieve only the repository metadata, branches, commit history and source code required to analyze a connected public repository. The system shall not modify repository contents, branches, settings or other repository resources.
SEC-06 [v2]	Repository Access Revocation	When authorization to a private repository is revoked, the system shall cease using that authorization to retrieve repository data or initiate new analyses of the affected repository.
SEC-07	Protection of Data in Transit	Communication containing authentication information, repository data, user data, or analysis data between users, system components, and external services shall be encrypted using secure transport protocols.
SEC-08	Protection of Sensitive Data at Rest	Authentication credentials, repository access tokens, API credentials, and other sensitive information persisted by the system shall be stored using secure mechanisms that protect them against unauthorized disclosure.
SEC-09	Protection of Authentication Secrets	Passwords, authentication tokens, API keys, private keys, and other authentication secrets shall not be exposed through user interfaces, analysis results, application logs, or user-facing error messages.
SEC-10	Session Security	Authenticated sessions shall be invalidated when the user logs out or when the associated authorization is revoked. Session credentials shall not remain valid beyond their configured expiration period.
SEC-11	Input Validation	The system shall validate externally supplied input before processing it and shall reject invalid, malformed, or unauthorized input without compromising system operation, security, or data integrity.
SEC-12	Protection Against Request Abuse	The system shall restrict excessive or abusive requests to externally accessible endpoints so that a single user, tenant, or external client cannot consume system resources in a manner that prevents normal service to other users.
SEC-13	Analysis Processing Isolation	Analysis jobs shall be processed in a manner that prevents repository data, intermediate analysis data, or analysis results belonging to one tenant from being accessed by users or analysis processes belonging to another tenant.
SEC-14	Security Event Logging	The system shall record security-relevant events, including failed authentication attempts, authorization failures, changes to repository authorization, and privileged administrative

CodeSage AI	Version: 1.0
Software Requirement Specification	Date: 09/08/2026
CS3203-G16-SRS-v1.0	

ID	Requirement	Verification Criteria
		actions, with sufficient information to support auditing and incident investigation.
SEC-15	Protection of Security Logs	System logs shall not contain plaintext passwords, repository access tokens, API keys, private keys, or other authentication secrets.
SEC-16	Secure Error Handling	User-facing error responses shall provide sufficient information to identify the failed operation without exposing authentication credentials, stack traces, database details, internal service addresses, or other sensitive implementation information.

3.6 Supportability

Supportability requirements state whether the system is maintainable when new maintainer joins the project and that he can change the system safely.

3.6.1 Coding standards and conventions

Table 3.6-1: Coding standards and conventions.

ID	Requirement
SP-1	The frontend shall be written in TypeScript with strict mode enabled and the backend and machine-learning code shall follow PEP 8 format.
SP-2	Commit messages shall follow the Conventional Commits convention and branches shall be named by type (feat/, fix/, docs/, test/).
SP-3	Every change shall reach the main branch through a reviewed pull request; unit tests and end-to-end tests shall be all green.

3.6.2 Modularity, configurability and extensibility

Table 3.6-2: Modularity, configurability and extensibility.

ID	Requirement
SP-4	A single data contract shall be crossing the frontend, backend and database boundary. It changes only through a pull request reviewed by developers from both backend and frontend.
SP-5	The codebase shall be layered (presentation, application/domain, data) within a monorepo. The presentation layer never accesses the database directly.
SP-6	Volatile third-party libraries shall sit behind exactly one boundary so that replacing one is a single-file change: all charts through one chart wrapper, the file-tree implementation behind one component, all HTTP through one API client module.
SP-7	Scoring shall remain a pure function over stored findings, so that changing a weight profile never requires rerunning a scan.
SP-8	Rule thresholds, severity base points and profile weights shall be configuration or data, never literals in code, so that calibration is a configuration change rather than a release.

CodeSage AI	Version: 1.0
Software Requirement Specification	Date: 09/08/2026
CS3203-G16-SRS-v1.0	

ID	Requirement
SP-9	All environment-specific values shall be supplied as environment variables (API base URL, mocking switch, database and broker URLs, GitHub credentials). No secret is committed to the repository and no environment difference requires a code edit.

3.6.3 Testability and diagnosability

Table 3.6-3: Testability and diagnosability.

ID	Requirement
SP-10	The frontend shall be fully testable without a backend: the same mock handlers serve development, unit tests and end-to-end tests. Every data hook and every dashboard component has a unit test, and the core workflows (dashboard render, scan, branch switch) have end-to-end tests.
SP-11	The rule engine shall be deterministic, so that regression tests are exact rather than statistical.
SP-12	System logs must be uniformly structured and explicitly linked using a unique scan identifier across all architectural components, including the API, the message broker, the worker nodes, and the machine-learning service, so that one scan is traceable end to end.
SP-13	The system must permanently record the final outcome of every scan directly into the main database, explicitly detailing both the final execution phase and any corresponding error messages. So that a user-reported failure is diagnosable from the database without inspecting logs.

3.6.4 Machine-learning, documentation and release maintainability

Table 3.6-4: Machine-learning, documentation and release maintainability.

ID	Requirement
SP-14	Models shall be trained offline and loaded at runtime as versioned artifacts, so that replacing a model requires no application change.
SP-15	Each stored snapshot shall record the model version that produced it. Without this, after re-training ML models, historical scores cannot be compared and the trend chart loses meaning.
SP-17	The system shall be extensible without engine changes: adding a rule is one threshold entry plus one reason template; adding a detector is a new source value, subject to FR-8.2: a source names a producer of findings, never a category or a file-level score; adding a language is a per-language rule pack plus recalibration.
SP-16	The risk model shall support periodic re-training on recent data with no architectural change.
SP-18	The SRS, the Software Architecture Document and application README files shall be maintained alongside the code, each stating the decision, its date and its rationale.
SP-19	To ensure system stability, the database will evolve strictly through incremental, versioned updates. Classifications like sources and categories will use flexible text identifiers tied to reference tables. Once established, existing identifiers cannot be changed, but new ones may be continuously added.

CodeSage AI	Version: 1.0
Software Requirement Specification	Date: 09/08/2026
CS3203-G16-SRS-v1.0	

ID	Requirement
SP-20	Every component shall ship as an independently buildable and independently scalable container, and builds shall be reproducible from committed lock files.

3.7 Design Constraints

The design of the system shall consider following constraints related to technical limitations, resource availability, security requirements and user requirements

Table 3.7-1: Design constraints.

ID	Constraint
DC-1	The machine learning component shall be trained using publicly available datasets, which require additional feature mapping and preprocessing to ensure compatibility.
DC-2	SATD classifier in the system shall initially focus on source code comments, excluding pull requests, issues, and commit messages as determining their current state is more complex.
DC-3	The system shall ensure the secure processing and storage of user repository information.
DC-4	The system shall require users to provide appropriate authorization.
DC-5	The system shall provide understandable analysis results, as users may have different levels of knowledge regarding technical debt concepts.
DC-6	The system shall provide documentation and guidance to help users interpret technical debt classifications.

3.8 On-line User Documentation and Help System Requirements

Table 3.8-1: Documentation and help requirements.

ID	Requirement
DOC-1	The system shall provide necessary documentation mentioned about connecting and managing GitHub repository.
DOC-2	The system shall provide descriptions of technical debt types and analysis result.
DOC-3	The system shall provide error messages and guidance in necessary situations.
DOC-4	The system shall provide about section with required information and version details.

3.9 Purchased Components

The system does not utilize any purchased component. All the framework, dataset, tools and libraries used in the system are freely available open-source resources.

3.10 Interfaces

3.10.1 User Interfaces

The graphical user interface is a single page web application. A persistent left navigation panel with icons provides navigation. Layout is described structurally; no screenshots are attached.

CodeSage AI	Version: 1.0
Software Requirement Specification	Date: 09/08/2026
CS3203-G16-SRS-v1.0	

Table 3.10-1: Global user-interface elements present on every authenticated screen.

Element	Required contents
Left navigation panel	Product name at top; navigation items; Projects, Dashboard, Scan History, Profiles can collapse to icons; the selected item is highlighted; an Account menu placed at the bottom.
Account menu	Sign out; Settings UI theme switch (dark/light) for v1.0
Notifications	Pop up messages for action outcomes: scan complete, scan failed, scan cancelled.

Table 3.10-2: Screen inventory: required panels, fields and states.

Screen	Required controls and fields	States
Login	A single "Sign in with GitHub" button, with the product name and tagline. No local username or password login.	Idle; authenticating; authentication error.
Projects	Connect panel: an example URL text box and a Connect button with inline validation. Project list (vertical): per row the repository name, owner, visibility(public/private), a health summary (score, grade, delta) only if a snapshot exists for a previous scan, and a Select action.	Loading; empty ("No repositories yet - connect one to see its health."); populated; invalid-URL error; unable to connect error.
Dashboard	Top navigation bar: active repository name, branch drop-down list, Scan control (Scan -> Scanning NN% with Stop -> idle), last-analyzed timestamp and short commit SHA. Left column: Overall Health card (score 0-100, grade A-E, delta, critical/high count, category pie), health-trend chart, and the Refactor-First list with a debt-type filter drop-down whose rows show category, severity badge, file: line and one-line reason. Right column: the hotspot file-tree heat map with expand, collapse, hover and select.	Loading; loaded; load error; scanning; scan error.
Dashboard - detailed mode	Selecting a finding replaces the health card and trend chart in place with the finding detail (evidence, one-line reason, file: line: symbol, and code snippet); the file tree auto-expands and highlights that finding's file; the Refactor-First list condenses, so the user can move between findings. Closing restores dashboard mode. The selected finding appears in the URL.	Entering; loaded; finding not found; leaving.
Scan History	A table of past snapshots: date, commit SHA, health score, grade, delta and finding count; selecting a row loads that snapshot into the dashboard read-only.	Loading; empty ("No scans yet"); populated.
Profiles	Three preset buttons (Balanced - the default - Security-first and Delivery-speed); five category weight sliders (security, code-design, requirement, documentation, test; clamped 0.1-3.0); one trust slider labelled at its ends "trust the rules" and "trust the model" (0-1, default 0.5); and a Reset to preset action. Applying a profile re-scores the stored findings instantly, with no re-scan.	Loaded; applying; modified-from-preset.

CodeSage AI	Version: 1.0
Software Requirement Specification	Date: 09/08/2026
CS3203-G16-SRS-v1.0	

User-interface conventions. Severity is always shown as a text badge. Grade and heat-map colors are always derived by numeric score, so that color is never the only carrier of meaning (U-8). All colors resolve from design tokens rather than hardcoded values (DC-2). All displayed values come from the data contract: the dashboard performs no calculation of its own. Every list defines a loading, an empty and an error state (U-14).

3.10.2 Hardware Interfaces

Code Sage AI is a browser-based web application and interfaces with no specialized hardware. No peripherals, sensors, printers or device drivers are used. The requirements below are client-side prerequisites and hosting expectations rather than hardware protocols.

Table 3.10-3: Hardware interfaces and client-side prerequisites.

Item	Requirement
Client device	Any desktop or laptop computer capable of running a current web browser. No installation and no local storage are required.
Client display	Minimum supported viewport 1280 x 720 (see U-13).
Client processor and memory	Dual-core processor and 4 GB RAM; the client performs rendering only, so requirements are modest.
Client network	Stable internet connection: the application is unusable offline.
Server / worker (hosting)	Standard cloud containers. Each worker requires local disk for a clone of the repository under analysis; at least 2 GB per concurrent scan, released on completion; outbound HTTPS access to the Git host.

3.10.3 Software Interfaces

Table 3.10-4: External software interfaces.

Interface	Purpose	Direction and protocol
GitHub OAuth	User authentication (FR-1).	Outbound; HTTPS, OAuth 2.0 authorization-code flow.
GitHub REST API	Repository metadata: name, owner, visibility, default branch, branch list and head commit SHAs.	Outbound, read-only; HTTPS with JSON, using ETag-based conditional requests to mitigate rate limits.
Git (clone)	Fetch working tree and the commit history for analysis, consume no API quota.	Outbound, read-only; Git over HTTPS.
Web browser runtime	Rendering host for the frontend. A Mock Service Worker (MSW) is only used for development and testing purposes, never in production.	Inbound; standard HTML, CSS and JavaScript.

CodeSage AI	Version: 1.0
Software Requirement Specification	Date: 09/08/2026
CS3203-G16-SRS-v1.0	

Table 3.10-5: Internal software interfaces.

Interface	Between	Protocol and format
REST API	Frontend and backend	HTTPS with JSON; payload formats are governed by the data contract; specified as an OpenAPI document in the Software Architecture / Design Document.
Task queue	Backend API and asynchronous workers	Redis protocol on a private network, carrying job enqueue and progress state.
Machine-learning inference	Worker and machine-learning service	Batched comments in, SATD label and category out; per-file numeric feature vector in, risk score 0-1 out.
Persistence	API and workers, and the database	SQL over TCP with TLS, using pooled connections.
Mock layer (development and test only)	Frontend and the mock service worker	Intercepts requests at the network boundary and adhere to the identical contract, so that moving to the real backend is a base-URL change rather than a rewrite.

Table 3.10-6: Application REST endpoints required for v1.0.

Method	Path	Purpose
GET	/api/auth/github	Begin sign-in. Redirects the browser to GitHub's authorization page with the requested scopes and a state value (FR-1).
GET	/api/auth/github/login	GitHub's redirect target. Exchanges the authorization code, establishes the session, and redirects the browser to the Projects page (FR-1).
GET	/api/auth/github/callback?code=&state= =	Return the signed-in user, or 401 if there is no valid session (FR-1).
POST	/api/auth/session	End the session and invalidate its credentials (FR-1, SEC-10).
GET	/api/projects	List the connected projects (FR-4).
POST	/api/projects	Connect a repository by URL (FR-3).
GET	/api/repos/{repoId}/branches	Branch list with head commit SHA and default branch flag (FR-5).
GET	/api/repos/{repoId}/health?branch= =	Full dashboard payload for one branch snapshot (FR-12 to FR-18). Scores are derived under the workspace's active profile.
GET	/api/repos/{repoId}/scans	Scan history: the stored snapshot summaries (FR-19).
POST	/api/repos/{repoId}/scan	Start a scan for a branch; returns a scan identifier and phase (FR-6).
GET	/api/repos/{repoId}/scan/{scanId}	Poll the phase and progress of a running scan (FR-6).

CodeSage AI	Version: 1.0
Software Requirement Specification	Date: 09/08/2026
CS3203-G16-SRS-v1.0	

Method	Path	Purpose
POST	/api/repos/{repoId}/scan/{scanId}/stop	Cancel a running scan (FR-6).
GET	/api/profiles	List the available scoring profiles (FR-20).
GET	/api/profiles/active	The workspace's active profile; category weights and the trust slider s. Seeds the Profiles on load (FR-20).
PUT	/api/profiles/active	Apply a profile. The body carries the complete profile (five weights and s); the server clamps each weight to 0.1–3.0, replaces the workspace's active profile, and returns the stored values (FR-20).

Datasets. The SATDAUG dataset and the D’Ambros defect dataset are offline. They are downloaded once per training artifact. No dataset and no external API is contacted during a scan.

3.10.4 Communications Interfaces

Table 3.10-7: Communications interfaces.

Channel	Protocol and format	Notes
Browser to frontend and API	HTTPS, JSON	All external traffic is encrypted; certificates are managed at the hosting edge.
Scan progress	Asynchronous HTTP polling of the scan-status endpoint	Polling interval is 1 second. No WebSockets or server-sent events in v1.0: polling is sufficient
Backend to Git host	HTTPS: REST for metadata and Git for clones	Read-only, least privilege.
Backend to workers	Redis protocol	Private network only; never exposed publicly.
Services to database	PostgreSQL wire protocol over TLS	Private network only.
Session and credentials	Bearer token supplied per request	Services remain stateless; tokens are never held in server memory.

3.11 Database Requirements

Table 3.11-1: Database requirements.

ID	Requirement	Description
DBR-1	Database Management System	The system shall use PostgreSQL as its primary relational database management system for storing the data of the system
DBR-2	Tenant Data Ownership	Any database record owned by a tenant, shall be associated with exactly one tenant. The association can be direct or through a parent record that belongs to the tenant.

CodeSage AI	Version: 1.0
Software Requirement Specification	Date: 09/08/2026
CS3203-G16-SRS-v1.0	

ID	Requirement	Description
DBR-3	Tenant Data Isolation	The database shall enforce a logical isolation between the tenants using PostgreSQL's Row-Level Security policies so that users and the system can access only data that belong to their authorized tenant.
DBR-4	Tenant and User Information	The database shall store workspaces, users and the membership relationship between them. In v1.0, each workspace shall be limited to one active member. However, the data model shall support multiple memberships per workspace in a later release.
DBR-5 [v2]	Role and Permission information	The database shall store the role and permission information required to support the role-based access-control behavior in the v2 release.
DBR-6	Analysis Attempt Records	The database shall save a record of every repository analysis attempt, including its tenant, repository, repository revision, trigger type, start time, completion time, execution status, retry count and failure information where applicable.
DBR-7	Repository Revision Tracking	Each analysis attempt shall be associated with an immutable repository revision identifier like a commit hash.
DBR-8	Analysis Engine Version	The database shall record the version of the analysis-engine that is used for each analysis attempt. The analysis-engine shall identify the applicable static-analysis tools, rule set, extraction logic and machine learning model versions used for each analysis attempt.
DBR-9	Repository Information	The database shall store metadata for each connected repository, including its source platform, external repository identifier, owner or namespace, repository name, repository URL, visibility, default branch, connection status and associated tenant.
DBR-10	Equivalent Analysis Comparison	The database shall support comparing the branch-head commit SHA of a requested analysis with the commit SHA of the repository's most recent successfully completed analysis.
DBR-11	Source-Code Component Metadata	The database shall store the metadata required to identify analyzed source-code components, including file paths, programming languages, function or method names, symbol names, and relevant source-code line ranges.
DBR-12	Extracted Software Metrics	The database shall store the software metrics extracted during each successful analysis and associate them with the relevant analysis attempt, repository, file, function or method.
DBR-13	Technical Debt Findings	The database shall store each and every detected technical debt finding together with its detector, rule or model, debt category, severity, description, affected source code component, confidence value where applicable and originating analysis.
DBR-14	Finding Source Locations	Each location specific finding shall be associated with the exact repository revision, relative file path and applicable start and end line and column positions. If a finding applies to an entire file, class, function or method, the database shall store the corresponding component-level location.

CodeSage AI	Version: 1.0
Software Requirement Specification	Date: 09/08/2026
CS3203-G16-SRS-v1.0	

ID	Requirement	Description
DBR-15	Finding Traceability	Each technical debt finding shall have a fingerprint that enables the system to determine if the finding is new, unchanged or resolved across successive analyses
DBR-16	Machine Learning Prediction Results	The database shall store the outputs produced by the SATD classification and bug-proneness prediction models such as predicted labels, confidence and explanations where applicable and the model version used.
DBR-17	Machine-Learning Model Registry	The database shall maintain metadata for each deployable machine learning model version such as model type, version identifier, training date, deployment status, evaluation dataset reference and evaluation metrics.
DBR-18	Analysis Provenance and Reproducibility	Each finalized analysis shall identify the repository revision and the versions of the analysis engine version used to generate its results. The database shall retain sufficient information to check the conditions defined in REL-10.
DBR-19	Prioritization Profile Storage	The database shall store tenant-configurable prioritization profiles including profile names, category weights, activation status creation time, modification time and the user responsible for each change.
DBR-20	Commit and Change History Data	The database shall store the minimum commit metadata required for historical analysis and churn calculation such as commit identifiers, parent relationships, branch references and time stamps.
DBR-21	Historical Analysis Data	The database shall preserve completed analysis results required to display code health trends, compare successive analyses, identify changes in technical debt and measure progress over time. These data shall support the derivation of repository health trends, finding changes and technical debt progress over time. Profile values such as finding priorities, file debt values, health scores and grades shall not be stored with historical analysis data.
DBR-22	Finalization of Analysis Results	Findings, extracted metrics, machine-learning predictions shall be atomically committed as a finalized result only after all required analysis stages have completed successfully. The database shall retain the status and diagnostic information of failed, cancelled or partially completed analysis attempts, but such attempts shall not be presented as finalized analysis results.
DBR-23	Preservation of Completed Results	The findings, extracted metrics, machine learning models' predictions, repository revision information of a completed analysis shall not be overwritten. Profile-dependent values shall be recalculated from the stored analysis facts when results are retrieved.
DBR-24	Referential and Domain Integrity	The database shall enforce primary-key, foreign-key, uniqueness, required-field and validity constraints to avoid orphaned, duplicated or structurally invalid records.
DBR-25	Transactional Consistency	Database operations that modify related persistent records shall be executed transactionally where partial completion could create inconsistent data.
DBR-26	Time Representation	All timestamps shall be stored using a consistent time zone-aware format, with UTC used as the internal time standard.

CodeSage AI	Version: 1.0
Software Requirement Specification	Date: 09/08/2026
CS3203-G16-SRS-v1.0	

ID	Requirement	Description
DBR-27	Repository Source-Code Non-Persistence	The database shall not store repository source-code files, complete file contents or arbitrary code snippets. For a comment classified as Self-Admitted Technical Debt the database may store the relevant comment text as evidence associated with the SATD finding. Only the comment that produced the finding shall be stored. Any unrelated source-code comments shall not be stored. The code snippets displayed in the finding-detail view shall be retrieved on demand using the stored repository revision and finding location and shall not be stored in the database.
DBR-28	Data Deletion and Dependent Records	The database shall support the authorized deletion of tenant, user, repository, configuration and analysis data while enforcing tenant ownership and role-based permissions. Deletion operations shall remove or appropriately anonymize dependent records without leaving orphaned or inconsistent data.
DBR-29	Protection of Sensitive Data	Authentication credentials, repository access tokens, API credentials and other sensitive information persisted in the database shall be protected against unauthorized disclosure and shall not be stored in plaintext.
DBR-30	Security Audit Records	Security audit records persisted in the database shall include the event type, timestamp, tenant, acting user or system identity, affected resource and outcome. Audit records shall not contain plaintext passwords, repository access tokens, API keys, private keys or other authentication secrets.
DBR-31	Backup and Recovery	The database shall be included in the system's backup and recovery procedures to protect tenant information, user information, repository metadata, configurations, analysis history and finalized analysis results in accordance with REL-07 and REL-09.
DBR-32	Database Query Performance	The database schema, indexes and query mechanisms shall support the retrieval of dashboard summaries, prioritized findings, file-tree data, analysis histories and trend information within the response-time requirements defined in the Performance Requirements section.
DBR-33	Baseline Database Capacity	The database shall support the workspace, user, repository, configuration, analysis-history, metric, finding and scoring data generated under the baseline system capacity defined in PERF-06.
DBR-34	Schema Extensibility	The database schema shall support the addition of new technical-debt categories, detectors, software metrics, machine-learning model versions and scoring mechanisms without invalidating previously stored analysis results.
DBR-35	Schema Versioning and Migration	Changes to the database schema shall be versioned and applied through controlled migration procedures that preserve existing valid data.

3.12 Licensing, Legal, Copyright, and Other Notices

CodeSage AI	Version: 1.0
Software Requirement Specification	Date: 09/08/2026
CS3203-G16-SRS-v1.0	

Table 3.12-1: Licensing and legal requirements.

Component	License / Copyright	Purpose
CK	Apache License 2.0 [1]	Code metrics for Java code by means of static analysis.
PyDriller	Apache License 2.0 [2]	Git repository mining and commit history extraction.
Tree-sitter	MIT License [3]	Source code parsing and comment extraction.
Scikit-learn	BSD 3-Clause License [4]	Machine learning model development.
FastAPI	MIT License [5]	Backend API development.
Redis (Redis $\leq$ 7.2 or alternatively Valkey)	BSD 3-Clause [6]	Background task queue management and message brokering with Celery.
Celery	BSD License [7]	Asynchronous task processing.
PostgreSQL	PostgreSQL License [8]	Database management.
Next.js	MIT License [9]	Frontend application framework.
Tailwind CSS	MIT License [10]	User interface styling.
D'Ambros	CC BY 4.0 [12]	Per-file bug prediction model training.
SATDAUG	Apache License 2.0 [13]	SATD code comment classification model training.

3.13 Applicable Standards

Table 3.13-1: Applicable standards.

ID	Standard and applicability
ST-1	OpenAPI 3.1: Defines the standard for specifying and documenting the REST API communication between the backend and frontend.
ST-2	OAuth 2.0 / RFC 6749: Governs the standards for secure user authentication, authorization, and data extraction.
ST-3	PEP 8: The required coding standard for all Python codebase development to ensure maintainability, readability, and simplicity.
ST-4	ISO/IEC 25010: Governs the software quality models used for performance evaluation, validation, and testing of the system and models.
ST-5	WCAG 2.1 AA: Defines the web accessibility standards that the frontend user interface must achieve.
ST-6	IEEE 830-1998: The standard to which this Software Requirements Specification (SRS) document is written and structured.

4 Supporting Information

The table of contents appears at the front of this document. The appendices below are part of the requirements where they define data shapes or traceability, and are informative where they record supporting material.

CodeSage AI	Version: 1.0
Software Requirement Specification	Date: 09/08/2026
CS3203-G16-SRS-v1.0	

4.1 Appendix A - Requirements traceability

This matrix is what demonstrates that every proposal objective is realised and that every requirement is testable.

Table 4.1-1: Requirements traceability matrix (extract).

Requirement	Proposal objective	Release	Verification / test case
FR-1 Authentication and session	Objective 3 - centralized interactive dashboard	v1.0	TC-01 sign-in end-to-end test
FR-2 Workspace and tenant isolation	Objective 2 - secure multi-tenant architecture	v1.0	TC-02 row-level-security isolation test
FR-3 Connect a repository	Objective 3	v1.0	TC-03 connect valid and invalid URL
FR-6 Repository scan	Objective 3	v1.0	TC-06 scan lifecycle end-to-end test (start, progress, cancel, error, skip-if-unchanged)
FR-7.1 Extraction boundary	Objective 1 - analyse the impact of technical debt	v1.0	TC-07 same-SHA re-scan reproduces an identical snapshot
FR-8 / FR-8.1 Rule engine and severity register	Objective 5 - evaluate ML against the rule baseline	v1.0	TC-08 deterministic rule fixtures; register review
FR-9 / FR-9.2 SATD classifier and severity	Objective 4 - NLP classifier for self-admitted debt	v1.0	TC-09 held-out comment evaluation; marker-table unit tests
FR-10 Risk model	Objective 5	v1.0	TC-10 precision / recall / F1 / AUC report
FR-11 Scoring and prioritization	Objective 3	v1.0	TC-11 worked-example fixture; bound check that no low finding outranks a critical one in the same category
FR-12 to FR-19 Dashboard outputs	Objective 3	v1.0	TC-12 dashboard render and drill-in E2E
FR-20 Scoring profiles	Objective 3	v1.0	TC-20 preset seeds sliders, weights clamp, re-score without a scan
FR-21 Snapshot persistence	Objective 1	v1.0	TC-21 append-only insert; trend and history reads
FR-24 Visibility floor	Objective 3	v1.0	TC-24 critical security finding stays visible at the minimum security weight
FR-25 ML evaluation	Objective 5	supporting	Model evaluation report, testing phase

CodeSage AI	Version: 1.0
Software Requirement Specification	Date: 09/08/2026
CS3203-G16-SRS-v1.0	

4.2 Appendix B - Dashboard output definitions

The six dashboard outputs - the overall health card, the category breakdown, the health trend, the Refactor-First list, the finding-detail panel and the hotspot file tree - are defined by the shared data contract that lives in the OpenAPI contract, which is the single source for their field names, types and permitted values. Any change to those shapes is a change to this appendix.

Table 4.2-1: Dashboard Definitions.

Output	Requirement	Contract object
Overall Health card	FR-12	HealthReport: health_score, grade, delta, red_issue_count
Category breakdown	FR-13	CategoryBreakdownItem []
Health trend	FR-14	HealthPoint []
Refactor-First list	FR-15	Finding []
Finding detail	FR-17	Finding
Hotspot file tree	FR-18	TreeNode[], FileScore[]

4.3 Appendix C - Rule register (rule id, severity, category, message template)

This appendix is the single normative source for two requirements at once: FR-8.1, which states that each rule carries a fixed severity and category, and FR-16, which states that each rule carries a message template. One row per rule closes both. The severity column of this table is the answer to "where does a finding's severity come from?".

4.3.1 C.1 Rule-engine rules (source = rule)

Table 4.3-1: Rule register: rule-engine rules.

Rule id	Category	Severity	Base	Message template
hardcoded-secret	security	Critical	8	A credential-like value is assigned to {symbol} - move it to an environment variable and rotate the key.
sql-concat	security	High	5	SQL is built by string concatenation in {symbol}() - use a parameterised query.
complex-function	code-design	Medium	3	{symbol}() has cyclomatic complexity {value}, over the limit of {threshold} - split it into smaller functions.
long-method	code-design	Medium	3	{symbol}() is {value} lines long, over the limit of {threshold} - extract cohesive blocks into helpers.
deep-nesting	code-design	Medium	3	{symbol}() nests {value} levels deep, over the limit of {threshold} - use early returns to flatten it.
large-file	code-design	Low	1	{file} is {value} lines long, over the limit of {threshold} - consider splitting it by responsibility.

CodeSage AI	Version: 1.0
Software Requirement Specification	Date: 09/08/2026
CS3203-G16-SRS-v1.0	

4.3.2 C.2 SATD marker patterns (source = satd)

The category is predicted by ML-1; the severity comes from the marker matched in the comment text (FR-9.2). Patterns are evaluated high to low and the highest match wins.

Table 4.3-2: Rule register: SATD marker patterns.

Marker pattern	Severity	Base	Message template
\b(FIXME BUG XXX BROKEN DO\s*NOT\s*(SHIP MERGE))\b	High	5	Self-admitted defect: '{comment_text}' - classified as {predicted_category}.
\b(TODO HACK TEMP TEMPORARY WORKAROUND KLUDGE REFACTOR)\b	Medium	3	Self-admitted debt: '{comment_text}' - classified as {predicted_category}.
\b(NOTE REVIEW NIT IDEA QUESTION MAYBE)\b	Low	1	Self-admitted note: '{comment_text}' - classified as {predicted_category}.
(no marker matched - the classifier detected debt in prose alone)	Medium	3	Self-admitted debt: '{comment_text}' - classified as {predicted_category}.

Table 4.3-3: Debt-category taxonomy and dataset label mapping.

Category (product value)	Dataset label	Instances	Assigned by
code-design	code/design_debt	2,703	ML-1 and the rule engine
requirement	requirement_debt	2271	ML-1
test	test_debt	2635	ML-1
documentation	documentation_debt	2701	ML-1
security	(not present in the dataset)	-	Rule engine only - never predicted
(not a category)	non_debt	58,204	The negative class of the debt / not-debt decision

Table 4.3-4: Risk badge template.

Trigger	Template
risk_score shown on a file	High-risk file ({risk}): {indicators}. For example: "High-risk file (0.78): high complexity (WMC 18) and frequent change (14 commits/90d)."